

Intro to TypeScript

Filip Kaštovský

Why TypeScript?

- **Development Velocity:** Cheaper developers, faster iterations.
- **The Universal Language:** Runs natively in browser, server (Node/Bun/Deno), edge, and mobile.
- **Fullstack Code:** Share models, validation (Zod), and logic everywhere.

Why TypeScript? (AI & Ecosystem Synergy)

- **AI Capabilities:** There is a lot of typescript code on the web. AI can leverage this collective knowledge.

Speed vs. Maintainability

- **The Sweet Spot:** Some safety of strict languages (Rust, Java) at iteration speeds of dynamic languages (Python, JS).
- **Mature Tooling:** Great support across all IDEs through LSP.
- **"Shift-Left" ROI:** TypeScript shifts bug detection into the IDE.
 - IDE bug = pennies
 - QA bug = dollars
 - Production bug = thousands

When NOT to choose TypeScript

- **CPU-Intensive Tasks:** Use Rust, C++, or Go for databases or low-latency systems. JS is mostly single-threaded.
- **Throwaway Scripts:** The build step overhead might not be worth it for small scripts (not always applicable)
- **Learning Curve:** Advanced generics can be overwhelming for junior teams on tight deadlines.

Structural vs Nominal Typing

TypeScript uses **Structural Typing** (*Duck Typing*).

```
class Point {  
  x: number;  
  y: number;  
}  
class Vector {  
  x: number;  
  y: number;  
}  
  
// Valid in TS! (Fails in Java/C#)  
let p: Point = new Vector();
```

Rule: If an object has all required properties with matching types, it is compatible.

Type Inference

TypeScript is smart. You don't need to annotate everything.

```
// Inferred as 'number[]'
const numbers = [1, 2, 3];

// Return type inferred as 'string'
function format(name: string, age: number) {
  return `${name} is ${age} years old`;
}
```

Best Practice: Let TS infer types. Annotate function arguments and public APIs.

Union Types (|) - logical OR

```
type ID = string | number;

function printId(id: ID) {
  console.log(`Your ID is: ${id}`);
}
```

Intersection Types (&) - logical AND

```
type Draggable = { drag: () => void };  
type Resizable = { resize: () => void };  
  
type UIComponent = Draggable & Resizable;
```

Type Narrowing

Must **narrow** the type before accessing specific properties.

```
function processValue(val: string | number | number[]) {  
  if (typeof val === "string") {  
    return val.toUpperCase(); // 'val' is string  
  } else if (Array.isArray(val)) {  
    return val.length; // 'val' is number[]  
  } else {  
    return val * 2; // 'val' is number  
  }  
}
```

Custom Type Guards

Functions to help TypeScript narrow types.

```
interface Cat {  
    meow(): void;  
}  
  
interface Dog {  
    bark(): void;  
}  
  
// Return type predicate `pet is Cat`  
function isCat(pet: Cat | Dog): pet is Cat {  
    return (pet as Cat).meow !== undefined;  
}  
  
function handlePet(pet: Cat | Dog) {  
    if (isCat(pet))  
        pet.meow(); // TS knows this is a Cat  
    else pet.bark(); // TS knows this is a Dog  
}
```

Interfaces vs Type Aliases

Interface

- Can be merged (Declaration Merging).
- Better for public APIs and class contracts.

Type Alias

- Represents primitives, unions, and intersections.
- Cannot be merged.

Interfaces vs Type Aliases: Examples

```
// Interfaces: Declaration merging
interface Window {
  title: string;
}
interface Window {
  url: string;
}
// Result: { title: string; url: string; }

// Type Aliases: Unions
type StringOrNumber = string | number;
```

Generics

Reusable components that work with a variety of types.

```
function identity<T>(arg: T): T {  
    return arg;  
}  
  
const num = identity<number>(42);  
const str = identity("Hello"); // Type inference works here too!
```

Generic Constraints

Restrict the types passed to a generic using `extends` .

```
interface HasLength {  
    length: number;  
}  
  
function logLength<T extends HasLength>(arg: T): T {  
    console.log(arg.length);  
    return arg;  
}  
  
logLength("Hello"); // OK  
logLength([1, 2, 3]); // OK  
// logLength(42); // Error
```

Utility Types

Transform existing types.

```
interface User {  
  id: string;  
  name: string;  
  email: string;  
}  
  
type PartialUser = Partial<User>; // All optional  
type UserPreview = Pick<User, "id" | "name">;  
type UserWithoutEmail = Omit<User, "email">;  
type ReadonlyUser = Readonly<User>;
```

any, **unknown**, and **never**

- **any** : Opts out of type checking. Avoid it!
- **unknown** : Type-safe **any** . Must be narrowed before use.
- **never** : Values that never occur (e.g., unreachable code).

Exhaustive Checks with `never`

```
function assertNever(x: never): never {  
    throw new Error("Unexpected object: " + x);  
}  
  
function processShape(shape: "circle" | "square") {  
    switch (shape) {  
        case "circle":  
            return /* ... */;  
        case "square":  
            return /* ... */;  
        default:  
            // Errors if a new shape is added but not handled  
            return assertNever(shape);  
    }  
}
```

Mapped Types

Iterate over keys of an existing type.

```
type BooleanMap<T> = {  
  [Property in keyof T]: boolean;  
};  
  
interface Features {  
  darkMode: string;  
  newUser: number;  
}  
  
// { darkMode: boolean; newUser: boolean; }  
type FeatureFlags = BooleanMap<Features>;
```

Conditional Types

Ternary operator for the type system.

```
type NonNullable<T> = T extends null | undefined ? never : T;  
type A = NonNullable<string | null>; // type A = string
```

Combined with `infer`, extracts types from functions or promises.

Runtime Validation with Zod

TypeScript types are erased at runtime. **Zod** validates external data.

```
import { z } from "zod";

const UserSchema = z.object({
  name: z.string(),
  age: z.number().min(18),
});

type User = z.infer<typeof UserSchema>;
```

Runtime Validation with Zod (Cont.)

```
// Validate at runtime
const result = UserSchema.safeParse(fetchUserData());

if (result.success) {
  console.log(result.data.name); // Typed as 'User'
}
```

Questions?

Part 2: Ecosystem & Node.js

The Modern TypeScript Ecosystem

Roles are now split:

1. **Type Checking:** `tsc --noEmit`
2. **Transpiling/Bundling:** Vite, esbuild, or SWC (compiles in ms).

Result: Fast dev servers with background type safety.

Single-File Compilation

Tools like `@vercel/ncc` :

- Compiles Node.js projects into a single `.js` file.
- Bundles all dependencies.
- Removes massive `node_modules`.
- Keeps Docker images small.

Execution & Runtimes

- `tsx`: Fast drop-in replacement for Node.js execution.
- **Node.js (v22+)**: Native experimental support for `.ts` files!
- **Deno & Bun**: Alternative runtimes where TypeScript is native—no build step.

How Node.js Works

- **V8 Engine:** Compiles and executes JS/TS.
- **libuv:** Provides asynchronous I/O based on event loops and thread pools.
- **Bindings:** Connects JS to C++ APIs.

Single-threaded for JS, but uses a background thread pool for blocking OS operations.

The Event Loop

Mechanism for non-blocking I/O.

Phases:

1. **Timers:** `setTimeout()`, `setInterval()`.
2. **Pending Callbacks:** Deferred I/O callbacks.
3. **Poll:** Retrieves new I/O events.
4. **Check:** `setImmediate()`.
5. **Close Callbacks:** `socket.on('close', ...)`.

Don't Block the Event Loop!

```
// BAD: Blocks all other requests!  
app.get("/heavy", (req, res) => {  
  let sum = 0;  
  for (let i = 0; i < 1e10; i++) sum += i;  
  res.send({ sum });  
});
```

Solution: Use **Worker Threads** for CPU-bound tasks.

Other TypeScript Tooling

- **DefinitelyTyped** (`@types/*`): Type definitions for untyped JS libraries (e.g., `npm i @types/lodash`).
- **JSDoc**: Type-check pure `.js` files using JSDoc comments and `// @ts-check` .

Packages: Creating and Deploying

Publishing to `npm` requires compiling to `.js` and `.d.ts`.

```
{  
  "name": "my-cool-package",  
  "main": "dist/index.js",  
  "types": "dist/index.d.ts"  
}
```

Monorepos & Workspaces

Host multiple apps and packages in one repository.

- **Workspaces** (pnpm, npm): Link local packages.
- **Build Tools** (Turborepo, Nx): Cache builds and run parallel tasks.

TypeScript Project References

Keep `tsc` fast in a monorepo by compiling incrementally:

```
{  
  "files": [],  
  "references": [{ "path": "./packages/shared" }, { "path": "./apps/frontend" }]  
}
```

Linting & Formatting

Traditional

- `typescript-eslint` : Checks for logical errors.
- Prettier: Handles formatting.

Modern: Biome

- Fast tool written in Rust.
- All-in-one formatter and linter.
- Zero configuration by default.

Strict Mode

Always enable `strict` mode for new projects!

```
{  
  "compilerOptions": {  
    "strict": true,  
    "noImplicitAny": true,  
    "strictNullChecks": true  
  }  
}
```

Prevents `null` and `undefined` errors at compile time.

Questions?

Further Reading:

- [TypeScript Handbook](#)
- [Type Challenges](#)